

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

Лабораторная работа 2

По дисциплине Объектно-ориентированное программирование

Тема работы Алгоритм генетический

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Иванов С.Е.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 РЕЗУЛЬТАТ РАБОТЫ.....	4
2 КОНТРОЛЬНЫЕ ВОПРОСЫ	9
ЗАКЛЮЧЕНИЕ	10

ВВЕДЕНИЕ

В отчёте представлено решение лабораторной работы. В тексте работы представлены результаты реализации алгоритмов и ответы на контрольные вопросы.

Цель — изучить генетический алгоритм и реализовать алгоритм средствами ООП для решения Диофантова уравнения.

Задание на лабораторную работу

Реализовать средствами ООП генетический алгоритм для решения задачи: найти решение Диофантова (только целые решения) уравнения: $a + 2b + 3c + 4d = 30$, где a , b , c и d - некоторые положительные целые.

1 РЕЗУЛЬТАТ РАБОТЫ

Для реализации поколения был написан класс «Generation». На рисунке 1 определены его атрибуты: область допустимых значений переменных, функция, искомый результат функции; реализован конструктор и getter для получаемого решения.

```
1  class Generation
2  {
3      public int min_arg_value = 1;
4      public int max_arg_value = 30;
5
6      public static Random r = new Random();
7      public Func<int[], int> f;
8      public int[] args;
9      public int ex_res;
10     private int? real_res;
11
12     public Generation(Func<int[], int> f, int[] args, int ex_res)
13     {
14         this.f = f;
15         this.args = args;
16         this.ex_res = ex_res;
17     }
18
19     public int RealResult
20     {
21         get
22         {
23             if (real_res == null)
24             {
25                 real_res = f(args);
26             }
27             return (int)real_res;
28         }
29     }
30
```

Рисунок 1 — Атрибуты и конструктор «Generation»

На рисунке 2 представлена реализация скрещивания с кроссинговером двух поколений и мутаций случайного гена особи.

```
1  public static Generation NewGen(Generation p1, Generation p2)
2  {
3      int n = p1.args.Count();
4      int[] newArgs = new int[n];
5
6      int x = r.Next(1, 4);
7
8      for (int i = 0; i < n; i++)
9      {
10         newArgs[i] = i < x ? p1.args[i] : p2.args[i];
11     }
12
13     return new Generation(p1.f, newArgs, p1.ex_res);
14 }
15
16 public static Generation operator +(Generation p1, Generation p2)
17 {
18     return NewGen(p1, p2);
19 }
20
21 public void Mutation()
22 {
23     int n = args.Count();
24     int imposter = r.Next(1, n);
25     args[imposter] = r.Next(min_arg_value, max_arg_value + 1);
26 }
```

Рисунок 2 — Скрещивание и мутации

Также были реализованы методы для поиска коэффициента приближённости к решению поколения и расчёт коэффициента выживаемости популяции (рисунок 3).

```

1  public int GetAc()
2  {
3      return Math.Abs((int)(ex_res - RealResult));
4  }
5  public static double SurvivalKoef(List<Generation> generations)
6  {
7      double sum = 0;
8      foreach (Generation gen in generations)
9      {
10         sum += gen.GetAc();
11     }
12     return sum / generations.Count();
13 }

```

Рисунок 3 — Рассчёты коэффициентов

Основной класс программы был заполнен атрибутами (функция, результат и прочее). В основной функции создан набор из пяти популяций 4.

```

1  public static Random r = new Random();
2  public static int res = 30;
3  public static int n = 4;
4  public static Func<int[], int> f = (a) => a[0] + 2 * a[1] + 3 * a[2] + 2 *
    ↪ a[3];
5
6  static void Main(string[] args)
7  {
8      List<Generation> generations = new List<Generation>();
9      for (int i = 0; i < 5; i++)
10     {
11         int[] newArgs = { r.Next(1, 31), r.Next(1, 31), r.Next(1, 31),
            ↪ r.Next(1, 31) };
12         generations.Add(new Generation(f, newArgs, res));
13     }

```

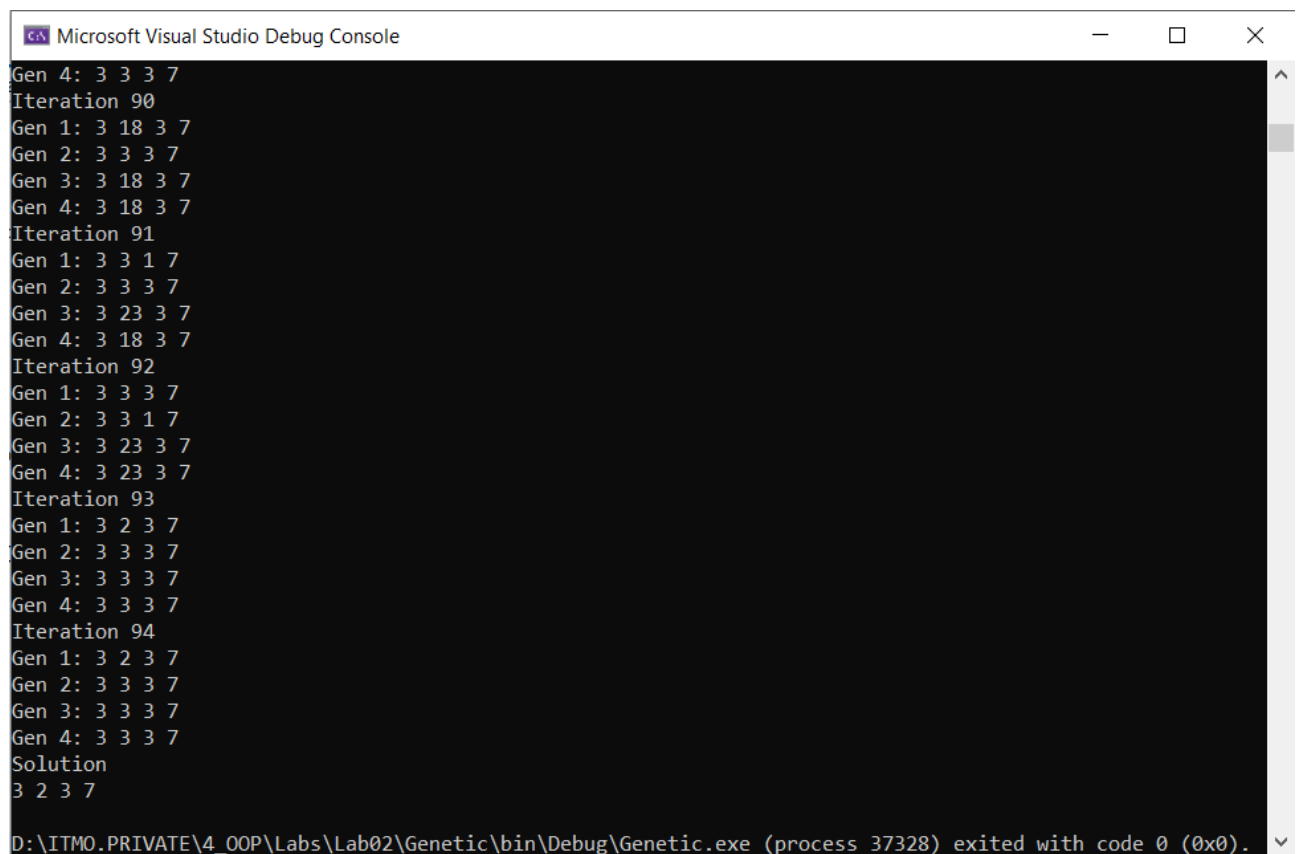
Рисунок 4 — Инициализация

На рисунке 5 показан основной цикл программы: проведение скрещивания и мутаций.

```
1  for (int i = 0; i<1000; i++)
2  {
3      Console.WriteLine("Iteration {0}", i + 1);
4      for (int j = 0; j < n; j++)
5      {
6          Console.WriteLine("Gen {0}: {1}", j + 1, string.Join(" ",
7              ↪ generations[j].args));
8      }
9
10     double koef1 = Generation.SurvivalKoef(generations);
11     var best = generations.OrderBy(g => g.GetAc()).ToList();
12     foreach (Generation gen in generations)
13     {
14         if (gen.RealResult == res)
15         {
16             Console.WriteLine("Solution");
17             Console.WriteLine(string.Join(" ", gen.args));
18             return;
19         }
20         generations = new List<Generation> {
21             best[0] + best[1], best[1] + best[0], best[0] + best[2],
22             best[2] + best[0], best[1] + best[2]};
23         double koef2 = Generation.SurvivalKoef(generations);
24         if (koef2 <= koef1)
25         {
26             int x = r.Next(1, 5);
27             for (int j = 0; j < x; j++)
28             {
29                 generations[r.Next(0, n)].Mutation();
30             }
31         }
32     }
```

Рисунок 5 — Итерации алгоритма

На рисунке 6 представлен результат работы программы.



```
Microsoft Visual Studio Debug Console
Gen 4: 3 3 3 7
Iteration 90
Gen 1: 3 18 3 7
Gen 2: 3 3 3 7
Gen 3: 3 18 3 7
Gen 4: 3 18 3 7
Iteration 91
Gen 1: 3 3 1 7
Gen 2: 3 3 3 7
Gen 3: 3 23 3 7
Gen 4: 3 18 3 7
Iteration 92
Gen 1: 3 3 3 7
Gen 2: 3 3 1 7
Gen 3: 3 23 3 7
Gen 4: 3 23 3 7
Iteration 93
Gen 1: 3 2 3 7
Gen 2: 3 3 3 7
Gen 3: 3 3 3 7
Gen 4: 3 3 3 7
Iteration 94
Gen 1: 3 2 3 7
Gen 2: 3 3 3 7
Gen 3: 3 3 3 7
Gen 4: 3 3 3 7
Solution
3 2 3 7
D:\ITMO.PRIVATE\4_OOP\Labs\Lab02\Genetic\bin\Debug\Genetic.exe (process 37328) exited with code 0 (0x0).
```

Рисунок 6 — Результат работы программы

2 КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Конструкции ООП: классы, свойства, методы, перегрузка операторов, делегаты, статические поля, коллекции, LINQ, инкапсуляция.
2. Оценка сложности: размер популяции $P = 5$, поколения $G = 1000$, длина хромосомы $L = 4$, фитнес $O(1)$, общая $O(G \cdot P \log P)$.
3. Решаемые задачи: поиск целых a, b, c, d ($1 \leq \dots \leq 30$) для диофантова уравнения $a + 2b + 3c + 2d = 30$.
4. Ограничения: представление хромосомой, фитнес-функция, диапазон значений, отсутствие гарантии точного решения.
5. Операторы: одноточечный кроссинговер, мутация заменой гена, элитарная селекция, наследование через потомков лучших.
6. Оценка точности: проверка совпадения с целевым значением, сравнение среднего отклонения (`SurvivalKoef`).

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы реализован генетический алгоритм на языке C#, позволяющий решить Диофантово уравнение, выбрав наилучшее решение.